효율적이고 신뢰할 수 있는 데이터 시스템을 만드는 엔지니어입니다.

Backend·Infra 2년 경험을 데이터 엔지니어링으로 확장

KEY SKILLS

Python SQL PostgreSQL Azure Databricks
Microsoft Fabric Go Docker

SOFT SKILLS

프로젝트 리딩 문제 정의·해결 부서 간 협업

Contents

PROJECTS

MS Data School | 프로젝트 3건 | 공통 역할: 팀 리딩·아키텍처 설계

프로젝트	역할	인원	기간	핵심기술	주요성과
취약점 재평가·조치 자동화 플랫폼	데이터·풀스택 엔지니어	6인	2026.08.03-09.04	Microsoft Fabric, FastAPI, PostgreSQL, Next.js	취약점 알림 **406→66→10건** 압축, 위험도 급등 취약점 **30건(8.7%)** 검증
오프보딩 이후 잔여 접근권한 탐지·조치 지원 플랫폼	데이터 엔지니어	5인	2026.06.22-07.09	Databricks, PySpark, ADLS Gen2, Unity Catalog	7개 시스템·16개 객체를 **4개 엔티티**로 통합, 주입한 오류 **37건 전부 탐지**
공공조달 입찰 요건 분석·대응 자동화 플랫폼	데이터·LLM 엔지니어	6인	2026.05.14-05.29	Azure Functions, Bicep, GitHub Actions	전체 모집단 대비 후속 처리 대상 **95.1% 축소**(15,894→779건), Azure 리소스 **8개** 배포 자동화

CAREER

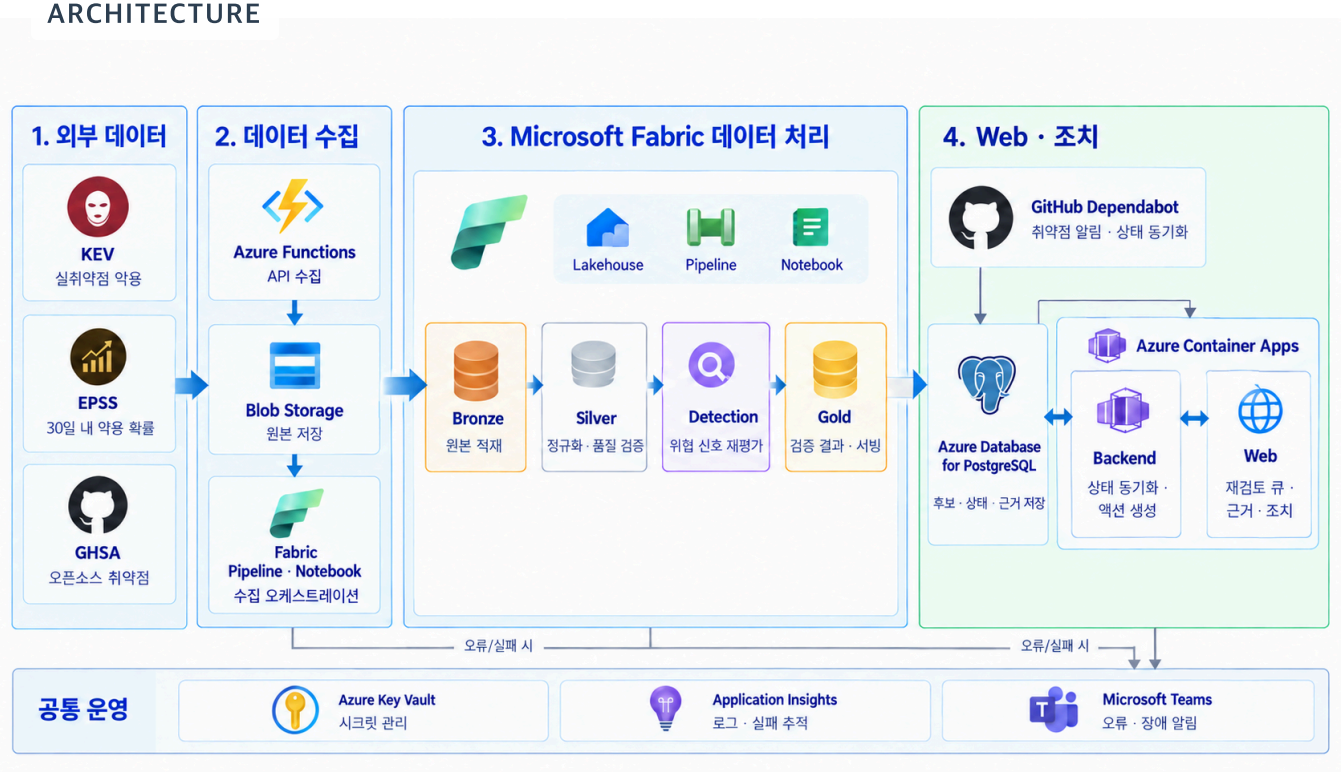
경력	역할	인원	기간	핵심기술	주요성과
이큐비알홀딩스	블록체인 연구원	4인	2021.01-2022.12	Go, MongoDB, AWS, Docker	**TPS 3,000→4,500** 향상, 백필 전송량 **60~70% 절감**

PROJECT 1

취약점 재평가·조치 자동화 플랫폼

2026.08.03-2026.09.04 | 데이터·풀스택 엔지니어 | Microsoft Fabric | FastAPI | PostgreSQL | Next.js

ARCHITECTURE



프로젝트 배경

- 취약점의 악용 가능성은 공개 이후에도 계속 변할 수 있음
- Dependabot 알림이 저장소·의존성 파일별로 분산돼 위험 분석 결과를 조치 단위로 연결하기 어려움

역할

- 소스별 갱신 특성에 맞춘 수집 전략 설계
- Fabric 취약점 분석 결과와 PostgreSQL 취약점 알림을 연결하는 동기화 구조 설계
- PostgreSQL 운영 DB와 FastAPI·Next.js 기반 재평가·조치 서비스 구현

핵심 결과

- EPSS 일별 위험도 이력 누적 **1.6억+건** 수집·보존
- 전체 취약점 **알림 406건 → 위험변화 매칭 66건 → 검토·조치 단위 10건**으로 단계적 압축

PROJECT 1 · 핵심 챌린지

소스별 갱신 특성에 맞춘 수집 전략 설계

EPSS
매일 갱신되는 악용 가능성 예측 점수(향후 30일 기준)

GHSA
GitHub 보안 권고 — 취약점·심각도 점수·영향 패키지·패치 버전 제공

KEV
실제 악용이 확인된 취약점 목록

문제

EPSS·GHSA·KEV는 제공 방식과 갱신 주기가 서로 달라, 하나의 수집 방식으로는 API 비용·이력 누락·재현성 문제가 발생

해결

소스 특성에 맞춰 수집 단위와 갱신 감지 방식을 다르게 설계

01 EPSS 스냅샷 보존

EPSS는 같은 취약점이라도 날짜별 점수 변화 자체가 재평가 판단의 근거라, (취약점, 스냅샷 날짜)를 기준 단위로 삼아 일별 전체 스냅샷을 그대로 보존

02 GHSA·KEV 변경 감지

GHSA는 마지막 수집 이후 바뀐 항목만 Watermark 기준으로 증분 수집하고, KEV는 미러 저장소의 새 커밋 여부를 확인해 변경 이벤트를 생성

결과

- GHSA·KEV는 변경분만 반영해 **전체 이력 재수집 없이 최신 상태 동기화**
- 소스별 수집 기준점과 고유 키를 적용해 **동일 구간 재실행 시 중복 적재 방지**

PROJECT 1 · 핵심 챌린지

분석과 조치 단위를 구분한 데이터 모델링



문제

위험 분석은 취약점 단위, 알림 원본은 저장소 단위, 실제 조치는 저장소·의존성 파일 단위로 판단 기준이 다름

해결

분석·보존·검토·조치 단계마다 다른 Grain을 쓰되, 각 Grain을 명시적으로 연결하는 모델 설계

01 Grain 분리 설계

취약점 ID로 위험 후보와 알림을 연결하고, 알림은 생태계·패키지 기준의 검토 그룹으로 통합. 각 그룹은 저장소·매니페스트와 연결해 실제 PR 조치로 연동

02 멱등 동기화·트랜잭션

취약점 수집·알림 동기화·매칭 각 단계에서 재실행과 부분 실패로 인한 중복 및 상태 불일치 방지를 위해, 고유 키 기반 멱등 처리·동시 실행 제어·단계별 트랜잭션 적용

결과

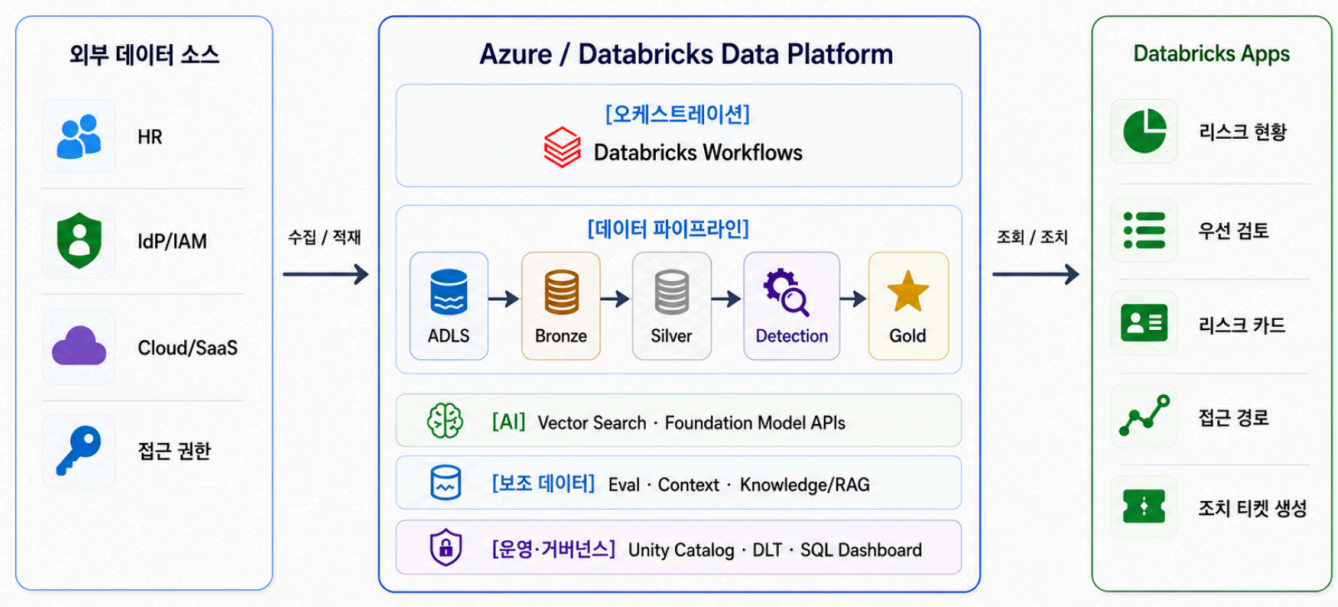
- 저장소·알림 원본을 그대로 보존하면서 검토는 패키지 단위로 통합 — **원본 추적성과 반복 검토 감소를 동시에 확보**
- 재실행·부분 실패에서도 **중복 없이 일관된 스냅샷 반영**

PROJECT 2

오프보딩 이후 잔여 접근권한 탐지·조치 지원 플랫폼

2026.06.22-2026.07.09 | 데이터 엔지니어 | Databricks | PySpark | ADLS Gen2 | Unity Catalog

ARCHITECTURE



프로젝트 배경

- 퇴사자 계정을 비활성화해도 자동화 계정·인증 정보가 별도 생명주기로 남아 간접 접근 경로가 생길 수 있음
- 시스템마다 계정·권한 표현 방식이 달라 이런 경로를 한 번에 확인하기 어려움

역할

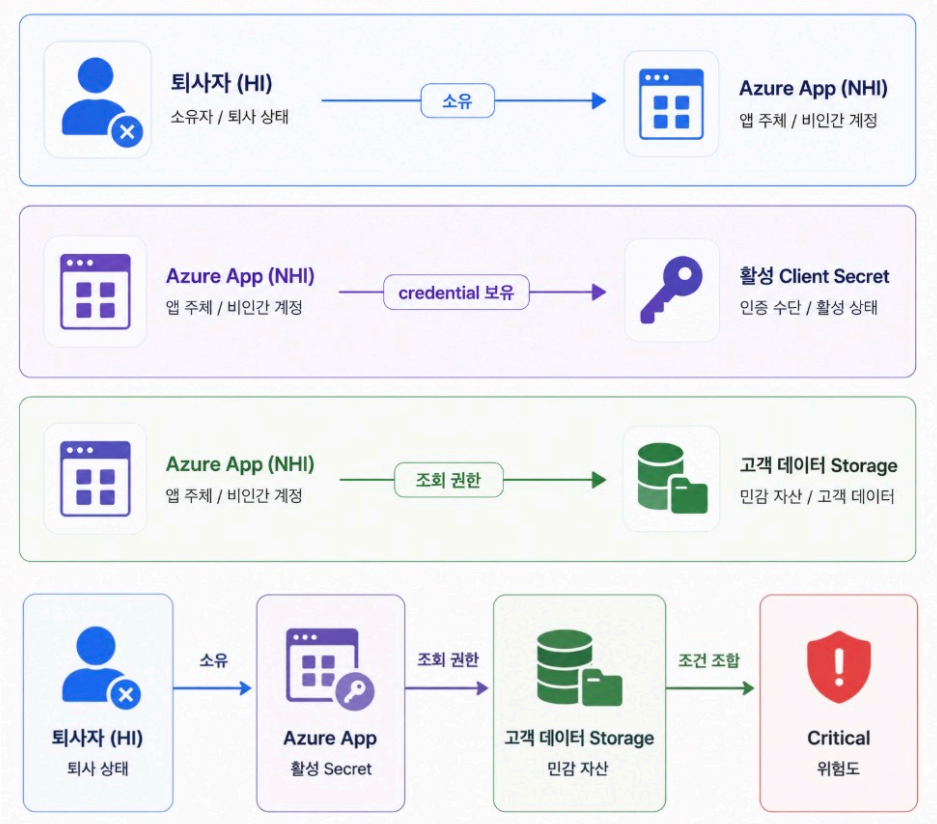
- 다중 시스템 계정·권한 데이터의 공통 스키마 설계
- 사람·비인간 계정의 잔존 접근 경로에 대한 위협 모델링
- 메달리온 ELT, DLT 품질·CI 검증 체계와 DAG 오케스트레이션 구현

핵심 결과

- **7개 시스템·16개 데이터 객체**를 4개 공통 엔티티로 통합
- Data Contract·CI 기반 품질 검증 체계로 스키마 불일치 사전 차단
- **12개 태스크** DAG로 오류 전파 제어

PROJECT 2 · 핵심 챌린지

접근 경로 모델링과 관계 무결성 확보



> **문제**
>
> 시스템마다 계정·권한 표현 방식이 달라, 퇴사자에서 자동화 계정과 인증 정보를 거쳐 민감 자산으로 이어지는 경로를 단순 계정 매핑으로 탐지하기 어려움

해결

접근 경로를 그래프 구조로 모델링하고, 관계의 의미까지 품질 게이트로 검증

01 공통 접근 모델

접근 주체·인증 정보·자산·관계 4개 공통 엔티티로 정규화해, '퇴사자 계정-자동화 앱-인증 정보-민감 자산' 등 복잡한 경로 표현

02 관계 의미 검증

소스 확장 과정에서 접근 시각·소유 관계가 시스템별로 다르게 생성되는 문제를 발견, 단순 null 체크만이 아닌 NHI→HI 소유처럼 허용되지 않은 관계를 품질 게이트에서 차단

결과

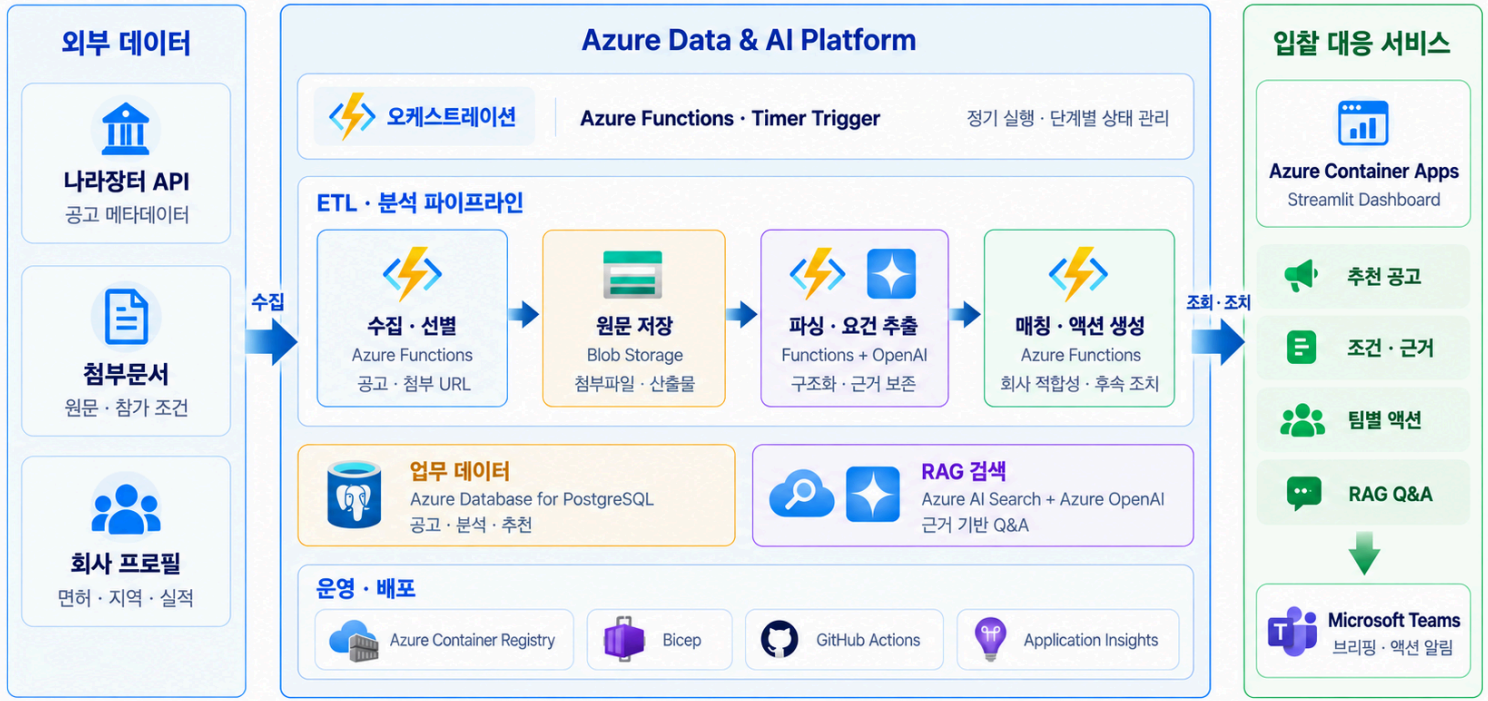
- **7개 시스템·16개 데이터 객체**를 4개 공통 엔티티로 통합, 직접·간접 접근을 동일 모델로 표현
- 의도적으로 주입한 스키마·데이터·관계 오류 **37건 전부 탐지**, 검증 실패 시 downstream 작업 실행 차단

PROJECT 3

공공조달 입찰 요건 분석·대응 자동화 플랫폼

2026.05.14-2026.05.29 | 데이터·LLM 엔지니어 | Azure Functions | Bicep | GitHub Actions

ARCHITECTURE



프로젝트 배경

- 나라장터 공고는 확인할 정보가 많고, 중요한 참가 요건은 첨부문서 안에 있어 공고만 보고는 즉시 판단하기 어려움
- 요건을 추출해도 실제 대응은 담당 부서별로 나눠 확인해야 해 검토가 분산됨

역할

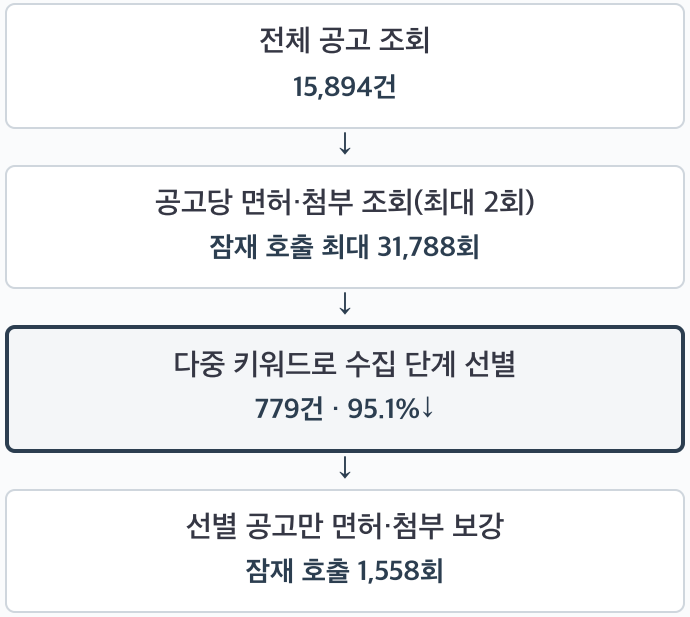
- 나라장터 공고·첨부문서 수집부터 요건 구조화·매칭·액션 생성까지 데이터 파이프라인 구현
- LLM 기반 요건 추출, 신뢰성 있는 RAG 질의응답 구현
- Bicep·GitHub Actions 기반 인프라 배포 자동화

핵심 결과

- 전체 모집단 대비 후속 처리 대상 **95.1% 축소**(15,894건→779건)
- LLM·규칙 기반 검증으로 입찰 조건 오분류 완화
- **8개 Azure 리소스** 구성·배포 표준화

PROJECT 3 · 핵심 챌린지

수집 단계 필터링을 통한 후속 처리 증폭 방지



문제

전체 공고 수집 시 검토 대상이 아닌 공고까지 면허·첨부정보 조회가 발생해 후속 작업이 입력 건수에 비례해 증폭

해결

검색 조건을 수집 단계에서 필터링해 선별된 공고만 후속 처리하는 구조로 전환

01 워크로드 비용 구조 고려

목록 조회보다 공고별 면허·첨부정보 조회·DB 처리 비용이 더 크다는 걸 확인 — 표본 100건 중 실제 후보 0건이었지만, 후보 여부와 관계없이 보조 API가 단일 건마다 호출되는 구조

02 수집 단계 필터링 키워드 보강

'시설관리' 공고가 단일 표현을 쓰지 않아 '시설관리·건물관리·청사관리' 등 18개 키워드를 API 검색 조건에 반영하고, 공고번호·차수 기준 중복 제거 후 선별된 공고만 후속 처리로 전달

> 고려한 대안
> 전체 수집 후 선별하는 순서도 검토했으나, 선별 단계도 15,894건 전체를 다뤄야 해 수집 단계 필터링 쪽으로 판단

결과

- 전체 조회 모집단 15,894건 대비 후속 처리 대상을 779건으로 **95.1% 축소**
- 공고별 보조 API 잠재 호출 규모 **최대 31,788회→1,558회**로 축소, API 한도 위험도 부수적으로 완화

CAREER

프로덕션 데이터 흐름과 운영 환경 개선

(주)이큐비알홀딩스 | 2021.01-2022.12 | 블록체인 연구원

데이터 서빙 구조 개선

- 합의 노드에서 원장 복제 노드를 분리하고 CQRS 기반 RDB 서빙 레이어 구축
- 원장 데이터를 실시간 스트림으로 적재하고 순차 정수 인덱스로 조회 경로 개선
- gzip 압축 백필과 이벤트 재생 기반 상태 재구성 구현

코어 전환과 데이터 호환

- PHP 기반 레거시 시스템을 검증된 Go 오픈소스 클라이언트 기반으로 전환
- 기존 데이터를 신규 클라이언트에서 처리할 수 있는 호환 구조 구현
- 합의 관련 데이터·메시징 인터페이스를 자사 네트워크 요구사항에 맞게 확장

개발·운영 환경 자동화

- Docker Compose로 다중 P2P 노드의 빌드·구성·피어링 자동화
- 노드 연결·합의 상태를 확인하는 시각화 도구 개발
- Grafana 기반 통신 지연·DB 사용량 모니터링 환경 구축

60~70% 절감

약 100만 블록·수백 GB 원장 백필 전송량

TPS 3,000→4,500

Go 기반 코어 전환 및 인터페이스 확장

50%+ 단축

개발·테스트 환경 구성 시간